

Tezcat: A Deterministic, Provenance-Aware Laboratory for Experimental Research on Emergent Financial-Market Phenomena

Satvik Anand

Registration No. RA2311026010278

[Institutional affiliation not established in repository materials; placeholder retained deliberately]

Abstract—Financial markets exhibit emergent phenomena—heavy-tailed returns, volatility clustering, liquidity crises, bubbles, flash crashes, and deleveraging cascades—whose mechanistic origins are difficult to isolate from historical data alone, while stylized theoretical models abstract away the market microstructure through which such phenomena propagate. We present Tezcat, a deterministic, provenance-aware experimental laboratory in which researchers construct artificial markets from heterogeneous trading agents interacting through a genuine price–time–priority limit order book, perturb individual mechanisms under controlled conditions, execute replicated experiment designs, and reproduce every published number from content-addressed identities. Tezcat contributes a reproducibility architecture (immutable experiment versions, deterministic seed allocation, canonical serialization, hash-chained event logs, state and event hashes, artifact-only reporting, and replay), an operational execution layer with provably order-independent parallelism, a read-only external event-market intelligence layer over Kalshi and Polymarket with immutable checksummed datasets and hypothesis-generating event signatures, and a bridge that exports deterministic synthetic market worlds into the NautilusTrader backtesting engine for strategy evaluation under controlled market ecologies. In a 420-run leverage $\times$ liquidity factorial conducted entirely through the platform’s own research surface, the leverage effect on maximum drawdown was large and threshold-like (+0.113 at the extremes; 0/60 crashes at a 2 $\times$ leverage cap), while the pre-registered leverage $\times$ illiquidity interaction was *rejected* (+0.015, 95% CI [−0.023, +0.057]); forced-liquidation volume concentrated in *deeper* books, indicating that within this model ecology liquidity acts as a transmission channel for deleveraging rather than a simple cushion. Tezcat is a simulation laboratory for mechanism experiments; it does not predict real markets, and no claim in this paper should be read as a real-market forecast.

Index Terms—agent-based simulation, market microstructure, limit order book, reproducibility, provenance, emergent phenomena, leverage, liquidity, backtesting, prediction markets

I. INTRODUCTION

Financial markets are complex adaptive systems whose most consequential behaviors—crashes, liquidity evaporation, leverage spirals, regime shifts—are *emergent*: they arise from interactions among heterogeneous participants, microstructural rules, and feedback loops rather than from any single actor’s intent [1]–[3]. Three established methodological families each illuminate part of this problem while leaving a gap. Empirical analysis of historical data documents robust statistical regularities [4] but rarely supports mechanism-level causal attribution,

because the historical record is a single uncontrolled realization. Stylized theoretical models [5], [6] isolate mechanisms analytically but abstract away the order-book microstructure through which real disturbances propagate. Machine-learning forecasting systems optimize predictive accuracy but are structurally unsuited to controlled mechanism experiments: their objective is extrapolation, not intervention.

Tezcat occupies a fourth position: *controlled synthetic market experimentation*. The system provides an artificial market in which price is endogenous—formed exclusively by order flow through a price–time–priority central limit order book (CLOB)—populated by heterogeneous agent classes with adaptive memory, subject to scripted shocks, margin constraints, and regime-dependent behavior. Around this engine, Tezcat builds the apparatus of experimental science: pre-registered designs with control and treatment arms, mandatory replication, deterministic seed allocation, immutable content-addressed experiment identities, hash-chained event provenance, statistical analysis with multiplicity control, artifact-only reporting, and a `reproduce` verb that re-executes experiments and verifies every stored hash. The intent is that a hypothesis about a market mechanism becomes a falsifiable, replicable, and reproducible experiment—and that a rejected hypothesis is a first-class result.

Two boundary statements govern everything that follows. First, **Tezcat does not predict real markets**; it studies which mechanisms *can* generate observed classes of phenomena under explicit assumptions. Second, external market data (Kalshi, Polymarket) enters only as read-only research observations that inform experiment design and calibration targets; it never drives the simulation kernel, and the platform contains no trading, wallet, or order-routing functionality toward any real venue.

The contributions of this paper are:

- C1** a deterministic agent-based market ecology built around a genuine price–time–priority CLOB with reservation-based accounting and endogenous price formation (Sec. V);
- C2** a reproducibility architecture based on immutable experiment versions, canonical serialization, deterministic seed allocation, state/event hashes, hash-chained event logs, artifact manifests, and replay (Sec. VI);
- C3** a complete research workflow connecting specification $\rightarrow$ replication $\rightarrow$ statistical analysis $\rightarrow$ artifact-only report-

ing $\rightarrow$ reproduction (Sec. VII);

- C4 a custom scenario system that compiles user-defined market designs into the same canonical experiment machinery (Sec. VIII);
- C5 TradeOps, an operational layer providing resumable batches, worker parallelism with serial-equivalent results, retries, duplicate-execution guards, and explicit failure semantics (Sec. VIII);
- C6 a stylized-facts and calibration layer with enforced out-of-sample validation and honestly reported identifiability limits (Sec. XI);
- C7 a read-only external event-market intelligence layer (Kalshi, Polymarket) with immutable checksummed datasets and event signatures that seed synthetic experiments (Sec. IX);
- C8 a bridge exporting deterministic synthetic market worlds into NautilusTrader for strategy evaluation under controlled ecologies (Sec. X);
- C9 a container-first deployment architecture, with an honest classification of which components are implemented versus planned (Sec. XII);
- C10 empirical mechanism $\rightarrow$ phenomenon demonstrations, centered on a 420-run leverage $\times$ liquidity factorial whose pre-registered interaction hypothesis was rejected and which surfaced a non-monotonic liquidity effect (Sec. XIV).

We claim novelty for the *combination* (Sec. III), not for each component individually.

II. BACKGROUND AND RELATED WORK

A. Agent-Based Computational Finance

The Santa Fe artificial stock market [1] established that heterogeneous, adaptively learning agents can endogenously generate market phenomena that representative-agent equilibria cannot. Lux and Marchesi [7] showed that interaction between chartists and fundamentalists produces scaling and volatility clustering consistent with empirical data. Surveys by LeBaron [8], Tesfatsion [9], and Hommes [3] consolidate this tradition, and Farmer and Foley [2] argue for agent-based modeling as policy-relevant scientific infrastructure. Gode and Sunder [10] demonstrated that even zero-intelligence traders interacting through a double auction generate meaningful market-level regularities—a caution against over-attributing structure to agent sophistication, which Tezcat inherits by including a noise-trader class.

B. Market Microstructure and Order Books

Kyle [5] and Glosten–Milgrom [6] formalize price formation under asymmetric information; Bouchaud *et al.* [11] document the statistical structure of real limit order books; Cont, Stoikov, and Talreja [12] model order-book dynamics as a stochastic queueing system; Chiarella and Iori [13] study double-auction microstructure by simulation. Tezcat adopts the simulation stance with an explicit operational CLOB: price–time priority, tick snapping, partial fills, order expiry, and declared self-trade semantics.

C. Stylized Facts

Cont [4] catalogs the empirical properties any credible market model should confront: absence of linear return autocorrelation, heavy tails, volatility clustering, and related regularities. Tezcat’s feature extractor (Sec. XI) follows this catalog directly.

D. Leverage, Liquidity, and Endogenous Instability

Geanakoplos [14] develops the leverage cycle; Adrian and Shin [15] document procyclical intermediary leverage; Brunnermeier and Pedersen [16] model mutually reinforcing market and funding liquidity spirals; Shleifer and Vishny [17] survey fire-sale dynamics. This literature motivates Tezcat’s margin engine, in which forced liquidation is *actual order flow* through the matching engine rather than an accounting adjustment—a design decision that proved decisive for the case study of Sec. XIV.

E. Reproducible Computational Science

Peng [18] and Sandve *et al.* [19] argue that computational claims require executable provenance; Boettiger [20] connects reproducibility to containerization. Tezcat operationalizes these principles for market simulation: content-addressed experiment identity, deterministic replay, and artifact-only reporting are enforced by the system rather than left to author discipline.

F. Market Simulation and Backtesting Infrastructure

ABIDES [21] is the closest system relative: a discrete-event multi-agent market simulator with realistic exchange messaging. Tezcat differs in emphasis rather than in kind: its core investment is the scientific workflow surrounding the simulator (immutable experiment objects, enforced replication, hash-verified reproduction, artifact-only reports) and the outward bridges to external event-market observation and strategy evaluation. NautilusTrader [22] is an event-driven backtesting/trading engine; Tezcat uses it strictly as a backtest-side consumer of exported synthetic worlds (Sec. X).

G. Prediction and Event Markets

Wolfers and Zitzewitz [23] and Arrow *et al.* [24] discuss prediction markets as information-aggregation mechanisms whose prices are informative but not identical to true event probabilities. Tezcat adopts this exact framing: Kalshi [25] and Polymarket [26] observations are labeled *market-implied probability proxies* throughout the system (Sec. IX).

Table I positions Tezcat against representative prior work without diminishing it: each cited line of research is stronger than Tezcat within its own aims.

III. RESEARCH GAP AND DESIGN REQUIREMENTS

Existing market simulators typically provide agent behavior, market dynamics, and visualization. The gap Tezcat addresses is *system-level*: no single open platform, to our knowledge,

TABLE I
LITERATURE POSITIONING (REPRESENTATIVE, NOT EXHAUSTIVE)

Work	Domain	Strength	Difference from Tezcat
Arthur <i>et al.</i> [1]; Lux & Marchesi [7]	agent-based finance	endogenous expectations; emergent scaling	no CLOB microstructure; no content-addressed reproduction workflow
Kyle [5]; Glosten–Milgrom [6]	micro-structure theory	analytic causal clarity	abstracted book; not an experimental platform
Cont <i>et al.</i> [12]; Bouchaud <i>et al.</i> [11]	order-book modeling	empirically grounded book dynamics	no heterogeneous behavioral ecology or experiment engine
ABIDES [21]	market simulation	high-fidelity exchange messaging	Tezcat adds immutable experiment identity, hash-verified reproduction, external-observation and strategy bridges
NautilusTrader [22]	backtesting engine	production-grade event-driven execution	consumes historical/synthetic data; does not generate controlled ecologies
Peng [18]; Sandve <i>et al.</i> [19]	reproducibility	principles and practices	Tezcat enforces them mechanically for market experiments

combines (i) operational CLOB microstructure, (ii) a heterogeneous behavioral ecology with adaptive memory, (iii) deterministic execution with immutable, content-addressed experiment identity, (iv) hash-chained event provenance with replay, checkpointing, and forking, (v) enforced statistical replication and multiplicity-controlled analysis, (vi) user-constructed scenarios compiling into one canonical execution path, (vii) read-only external event-market observation with dataset-level provenance, (viii) exportable synthetic worlds for strategy evaluation in an external backtesting engine, and (ix) an operational execution layer with fault-injection-tested storage—under a container-first deployment model. We claim the *combination* as the contribution; individually, most components have precedents cited above.

This gap translates into design requirements that recur throughout the paper: R1, price must be endogenous; R2, identical inputs must yield byte-identical outputs; R3, every published number must trace to a persisted artifact; R4, external data must never silently mutate the simulation kernel; R5, negative results must be as publishable as positive ones; and R6, the platform must remain runnable if every optional external integration is removed.

IV. TEZCAT SYSTEM ARCHITECTURE

Fig. 1 shows the implemented architecture; Table II inventories the subsystems. A React/Vite terminal dashboard and a research CLI drive a FastAPI control plane, beneath which sit the experiment registry, run manager, research API, and TradeOps queue. The deterministic ecology engine executes a per-step loop (shocks $\rightarrow$ regime evaluation $\rightarrow$ agent decisions $\rightarrow$ matching $\rightarrow$ margin/liquidation $\rightarrow$ snapshot/metrics/event-log append). The research layer provides batch execution, statistics, stylized facts, calibration,

TABLE II
SUBSYSTEM INVENTORY (AUDITED AT COMMIT 16448B4)

Subsystem	Role
Market engine	CLOB, price–time priority matching, partial fills, tick snapping, order expiry, declared self-trade policy
Agent ecology	five strategy classes with per-agent adaptive memory
Shock/regime engines	scripted whale/market-maker/sentiment shocks; stable/crisis/recovery detection with hysteresis
Portfolio/risk	reservation accounting; opt-in margin with liquidation as real order flow
Event log	hash-chained canonical log; replay reconstructs state
Registry & batch	immutable experiment versions; resumable replicated execution (serial $\equiv$ parallel)
Analysis & reports	bootstrap CIs, permutation tests, Holm correction; artifact-only rendering
Scenario builder & TradeOps	user scenarios compiled to canonical experiments; queued, guarded operational execution
External intelligence	read-only Kalshi/Polymarket adapters; immutable datasets; event signatures; research bridge
Worlds & strategy lab	deterministic market worlds; ecology fingerprints; NautilusTrader backtests
Research control plane	typed artifact graph over datasets, forecasts, portfolio decisions, worlds, backtests, risk, reports

replay, and hashing. Two outward-facing layers—external event-market intelligence (Sec. IX) and the synthetic-world / NautilusTrader bridge (Sec. X)—and a research control plane that links datasets, forecasts, portfolio decisions, worlds, backtests, and reports into one artifact graph, complete the system. Dependency direction is strictly *external systems* $\rightarrow$ *adapters* $\rightarrow$ *Tezcat*: the simulation kernel imports neither the market-data adapters, nor NautilusTrader, nor any optional desk dependency (requirement R6).

V. MARKET ECOLOGY ENGINE

A. Formal Setting

Let S_t denote market state at discrete step t , comprising the order book B_t , last price P_t , environment variables (sentiment, regime), and agent states $A_{i,t}$. Each agent draws an action from its policy conditioned on the public view and its private memory $m_{i,t}$:

$$a_{i,t} \sim \pi_i(S_t, m_{i,t}; \theta_i), \quad (1)$$

where θ_i are configured behavioral parameters. Submitted orders O_t transform the book through the matching mechanism

$$B_{t+1} = M(B_t, O_t), \quad (2)$$

with M implementing price–time priority with partial fills; P_t is the last trade price and is therefore *endogenous*: no external price series is supplied in normal synthetic experiments (R1). Agent wealth follows

$$W_{i,t} = C_{i,t} + P_t Q_{i,t}, \quad (3)$$

with cash C and inventory Q governed by reservation accounting: placing a buy (sell) order reserves cash (inventory), so no agent can commit resources twice; absent the margin extension, cash and inventory cannot go negative, and conservation invariants are checked on every run. Standard observables are computed per step: log return $r_t = \log(P_t/P_{t-1})$, rolling volatility σ_t , drawdown $DD_t = 1 - P_t/\max_{s \leq t} P_s$, spread

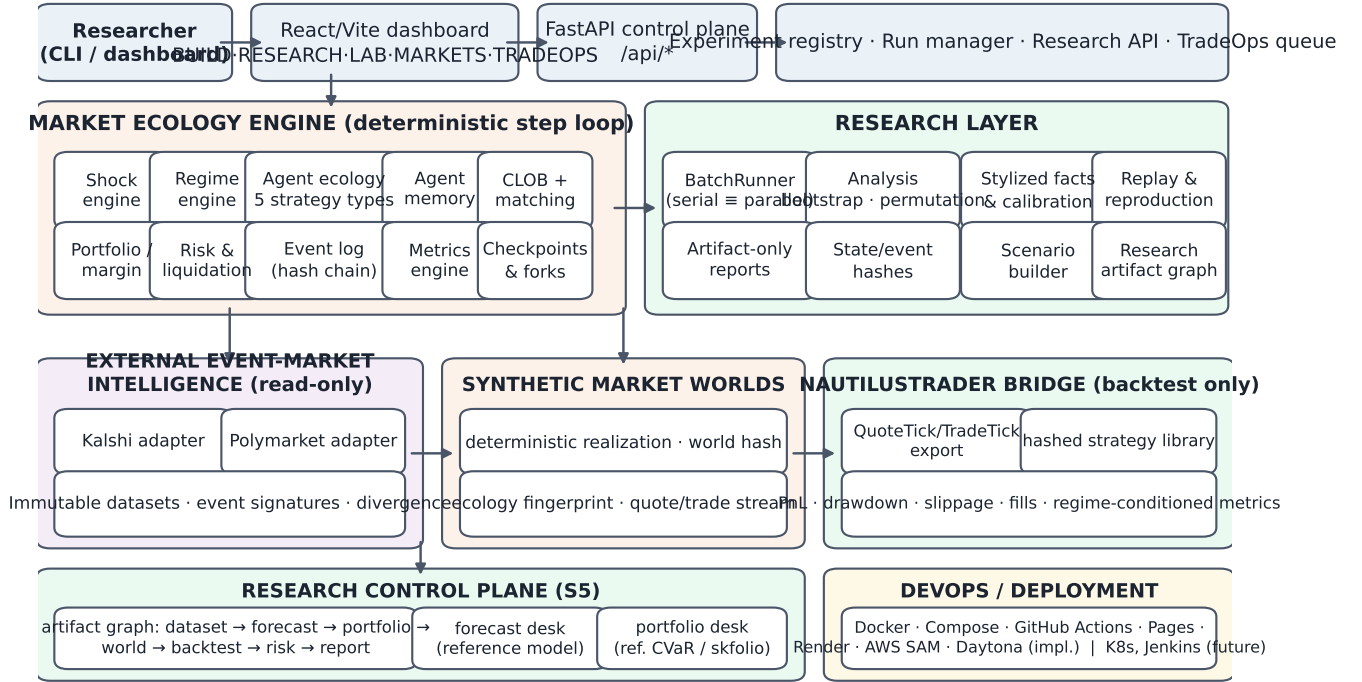


Fig. 1. Implemented Tezcat architecture. All layers exist in the audited repository; within the DevOps box, components are explicitly partitioned into implemented/configured versus future work (cf. Sec. XII).

TABLE III
AGENT CLASSES AND BEHAVIORAL MECHANISMS

Class	Mechanism
Noise	seeded random direction/size; liquidity background (cf. zero-intelligence benchmarks [10])
Retail	sentiment- and herding-driven; panic amplification under crisis regimes
Momentum	enters when short-horizon returns exceed a threshold; trend-following flow
Mean reversion	trades deviations from a fundamental anchor within a band; stabilizing flow
Market maker	two-sided quotes around mid with inventory-skewed pricing; withdraws under stress; primary endogenous liquidity supply

$s_t = P_t^{\text{ask}} - P_t^{\text{bid}}$, book depth, order imbalance OI_t , and an inventory-concentration HHI .

B. Agent Classes and Memory

Table III summarizes the five behavioral classes. Each agent additionally maintains a memory vector $m_{i,t}$ (fear, confidence, trend belief, value belief) updated from its own experience; losses raise fear and shrink participation and sizing, gains do the reverse. Regime state modulates behavior further (aggression, spread, participation, and cancellation multipliers).

C. Shocks, Regimes, Margin

Shocks are declared, scheduled interventions: whale order programs (large directional flow sliced over steps),

market-maker withdrawal (quoting suspension), and sentiment shocks (environment shift). The regime engine classifies stable/crisis/recovery from rolling drawdown, spread, depth, and volatility with hysteresis (persistence requirements prevent flapping), and regime events are recorded in the provenance log. The opt-in margin extension defines equity $E_{i,t} = C_{i,t} + Q_{i,t}P_t$ and margin ratio $E_{i,t}/(Q_{i,t}P_t)$; buys are admitted only above an initial margin im (leverage cap $\approx 1/im$), and maintenance breaches trigger—after a declared delay—a liquidation *process*: forced market sells in declared slices that pass through the matching engine like any other flow. Forced volume therefore measures *fills*, *not intent*; this distinction is central to Sec. XIV. Fig. 2 summarizes the interaction structure.

VI. REPRODUCIBILITY AND SCIENTIFIC PROVENANCE

A. Identity

All configuration is expressed in frozen, schema-versioned models serialized by a canonical JSON encoder (sorted keys, native types only; non-canonical values are rejected rather than coerced). An *ExperimentVersion* binds a base configuration, a validated design (question, hypothesis, variables, arms/factors, replications, budget), and a deterministic seed plan into a

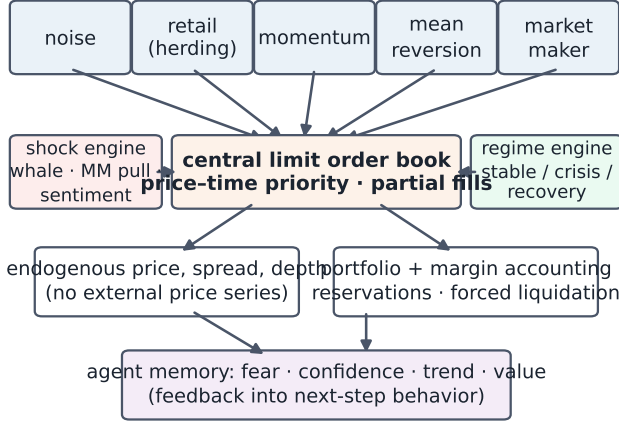


Fig. 2. Agent ecology and order-book interaction. Price, spread, and depth are endogenous outcomes of matched order flow.

content-addressed research identity:

$$H_{\text{exp}} = \text{SHA256}(\text{config} \parallel \text{design} \parallel \text{seedplan} \parallel \text{versions}), \quad (4)$$

where “versions” covers the config schema version, seed-allocator version, and code version. Registered versions are write-once; loading re-verifies the hash, so schema or code drift fails loudly instead of silently reinterpreting old records. Per-run seeds derive deterministically from the root seed, cell name, and replication index, making them independent of execution order and worker count.

B. Execution Provenance

Every run appends orders, trades, reservations, shocks, and regime transitions to a hash-chained event log; the run yields a state hash (canonical digest of terminal book/portfolio/environment state) and an event hash (digest over the chained log). Replay rebuilds exact state from events alone; checkpoints restore bit-identically mid-run, and forks declare interventions with full parent lineage. Artifacts (snapshots, trades, metrics, reports) are persisted with manifests, and reports are rendered *exclusively* from persisted artifacts—a test verifies that rendered numbers equal independently recomputed values (R3). Table IV summarizes the guarantees; Fig. 3 shows the pipeline. Later layers extend the same discipline outward: external datasets, market worlds, strategies, and research-plane artifacts each carry SHA-256 identities that chain back to H_{exp} .

C. The Frozen Baseline as Evidence

Development proceeded in audited phases. Phase F0 froze a seed-42 baseline of the three preset ecologies (51 passing tests) and recorded their outputs verbatim; every subsequent phase—semantics hardening (F1), identity/provenance (F2), events/replay, research objects, batches, analysis, microstructure, risk, stylized facts and calibration (F9), the research CLI/API/dashboard loop (F10), and parallel execution with

TABLE IV
REPRODUCIBILITY GUARANTEES (ALL ENFORCED BY TESTS)

Guarantee	Mechanism
Same inputs $\Rightarrow$ same run	seeded RNG; deterministic agent ordering; canonical config
Immutable experiment identity	content-addressed <i>ExperimentVersion</i> ; write-once registry; hash re-verified on load
Order/worker independence	per-run seeds derived from (root, cell, replication); serial $\equiv$ parallel byte-for-byte
Event provenance	hash-chained log; replay reconstructs state; checkpoint/fork lineage
Artifact-backed reporting	reports render only from persisted artifacts; equality test vs. recomputation
Chain reproduction	<code>tezcat reproduce</code> re-executes sampled runs and verifies seeds, state/event hashes, log chains, metrics
External/world/strategy identity	dataset hashes, world hashes, strategy hashes, environment records; version mismatch fails reproduction

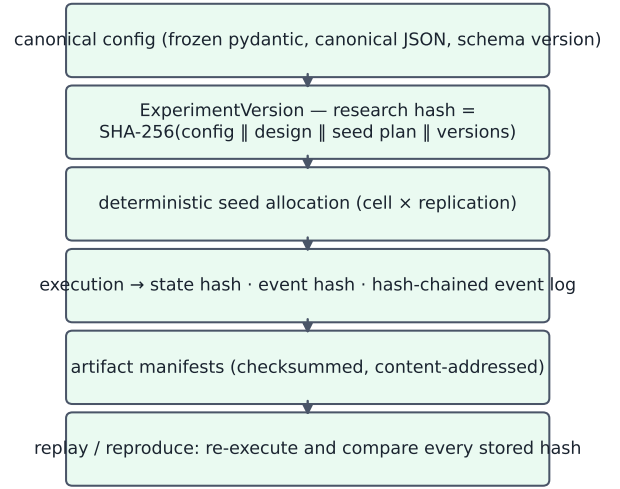


Fig. 3. Provenance pipeline from canonical configuration to hash-verified reproduction.

fault injection and differential validation (F11)—was required to keep that baseline byte-identical. The suite grew from 51 tests (F0) to 208 at the end of S1, then 231 (S2, scenarios and TradeOps), 335 (S3, external intelligence), 376 (S4, market worlds and the NautilusTrader bridge), and 415 at the audited commit (16448b4), where all 415 tests pass and the seed-42 determinism gate still reports identical outputs. A stronger demonstration appears in Sec. XIV: an experiment specification committed several development phases earlier re-minted its original research hash exactly when re-registered at the audited commit.

VII. EXPERIMENT DESIGN AND RESEARCH WORKFLOW

A design declares its type (baseline, A/B, ablation, sweep, or factorial), question, hypothesis, variables, primary metric, arms or factors, and replications; non-baseline designs *require* at least two replications (“one seed is a demonstration, not evidence”), and budgets cap planned runs with explicit, never

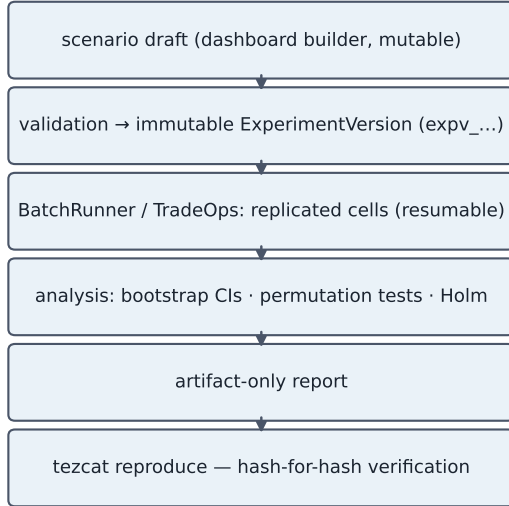


Fig. 4. Scenario-to-reproduction workflow. User scenarios compile into the same immutable experiment machinery used by presets and code-level specifications (one canonical execution path).

silent, truncation. The batch runner executes the design matrix resumably and idempotently; analysis provides seeded bootstrap confidence intervals, permutation tests with Holm correction, effect sizes, and 2×2 interaction contrasts; reports are artifact-only; and reproduction re-executes sampled runs against every stored hash. The complete loop is exposed identically through the CLI (`run / analyze / report / reproduce / list`), the research API, and the dashboard (Fig. 4).

VIII. CUSTOM SCENARIO BUILDER AND TRADEOPS

A. Scenario Builder

The dashboard scenario builder turns preset demonstrations into user-defined research: market configuration (tick, order-size and age limits), agent population and behavioral parameters, risk/margin parameters, shock timeline, and simulation length are edited as a mutable draft, validated by a dry run through the real *ExperimentVersion* constructor (surfacing the exact research identity the design would mint), and then registered immutably. Scenarios are *not* a second simulation path: no frontend simulation logic exists, and a compiled scenario is indistinguishable from a hand-written specification (C4).

B. TradeOps

TradeOps wraps batch execution in an operational state machine (`QUEUED` $\rightarrow$ `RUNNING` $\rightarrow$ `COMPLETED/FAILED/CANCELLED`) with worker processes, capacity guardrails, cooperative cancellation, identity-preserving retries, a duplicate-execution guard, and persisted partial results, so an interrupted batch resumes honestly rather than silently restarting. Parallel execution is required to be *byte-identical* to serial execution for any worker count; this

property is tested, not assumed. Fault-injection testing of the storage layer (F11) surfaced a genuine concurrency defect: atomic writes used a *shared* temporary filename per key, so two writers racing on the same key collided—one rename won and the other’s temporary file vanished. The fix (writer-unique temporary names incorporating process and thread identity) is now load-bearing across the registry, world, dataset, and artifact stores; we report the bug because it illustrates why fault injection belongs in scientific-infrastructure test suites.

IX. EXTERNAL EVENT-MARKET INTELLIGENCE

A. Read-Only Providers, Canonical Semantics

The S3 layer integrates Kalshi [25] and Polymarket [26] as *read-only observation sources*—not trading integrations: there is no order placement, account, wallet, or key functionality anywhere in the layer, and public market-data endpoints are used without credentials. Prediction-market semantics are preserved by a canonical schema ($\text{Event} \rightarrow \text{Market} \rightarrow \text{Outcome} \rightarrow \text{Observation}$) in which a YES price is always a *market-implied probability proxy* [23], [24], never a forecast or a true probability; provider-native units (Kalshi cent/dollar representations, Polymarket decimal strings) are preserved verbatim beside every canonical value, and each conversion names its rule. The Kalshi adapter covers events, markets, order books (YES/NO bid representation with the exact binary identity $\text{ask} = 1 - p^{\text{NO}, \text{bid}}$ recorded as the transform), trades, and candlestick history; the Polymarket adapter covers the Gamma (metadata), CLOB (books/prices), and Data API (trades) surfaces. Both adapters implement pagination, token-bucket rate limiting, retry with backoff, and strict schema-drift detection: a missing or renamed provider field raises a named error rather than being silently reinterpreted.

B. Immutable Datasets and Provenance

Every import follows raw response $\rightarrow$ immutable raw artifact $\rightarrow$ canonical normalization $\rightarrow$ registered dataset with SHA-256 checksums of both raw and normalized artifacts, adapter and schema versions, retrieval provenance, sampling declaration, and mandatory license/terms metadata with a permitted-use category (endpoint accessibility is never treated as redistribution permission). Datasets are content-addressed and write-once; quality violations (duplicates, timestamp regressions, out-of-range probabilities) reject a dataset rather than silently cleaning it. The test suite operates fully offline over labeled synthetic fixtures; no recorded provider data is bundled.

C. From Observation to Experiment

The research bridge (Fig. 5) turns an observed episode into a controlled experiment: a versioned, hashed *event signature* (pre/post implied probability, Δp , peak, time-to-peak, spread/volume/depth changes, and bounded-probability stylized features computed on Δp ; a logit transform is available but refuses boundary values rather than clipping) is extracted

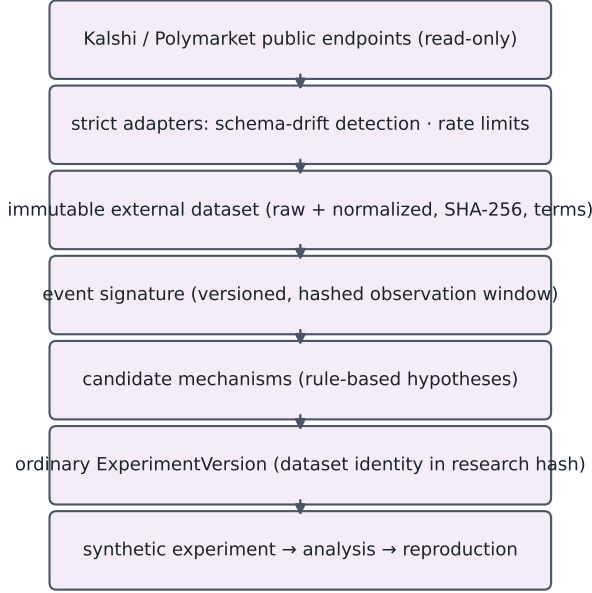


Fig. 5. External event-market intelligence: read-only observation to reproducible synthetic experiment. External data informs design and calibration targets; it never mutates the simulation kernel (R4).

over a declared window; rule-based templates propose *candidate mechanisms* (information shock, herding, momentum amplification, market-maker withdrawal, thin liquidity) phrased as hypotheses; and an ordinary *ExperimentVersion* is compiled whose research hash embeds the dataset identity, so reproduction verifies the experiment *with respect to that exact dataset version*. Cross-provider analysis is deliberately conservative: algorithmic event matching caps at “likely equivalent” (only human review with recorded rationale can assert exact equivalence), price differences are labeled *cross-provider probability divergence* rather than arbitrage, and lead/lag measurements are labeled observational with no causal claim.

X. SYNTHETIC MARKET WORLDS AND THE NAUTILUSTRADER BRIDGE

A *market world* is one deterministic realization of a registered experiment cell: (version, cell, replication) fixes the seed, the engine runs, and the result is captured as a canonical stream—L1 quotes with aggregate side depth as sizes (steps with an empty side are counted, never fabricated), trades with an honest NO_AGGRESSOR marker (the matcher does not record taker side), and regime/shock annotations with derived intervals—under a deterministic step-to-nanosecond time mapping. The world hash covers the research hash, seed, config/state/event hashes, stream checksums, time mapping, and a versioned *ecology fingerprint*: a purely numeric environment identity (relative spread, depth quantiles, imbalance, return volatility, Hill tail index, volatility clustering, drawdown/crash flag, trade intensity, regime occupancy, agent composition).

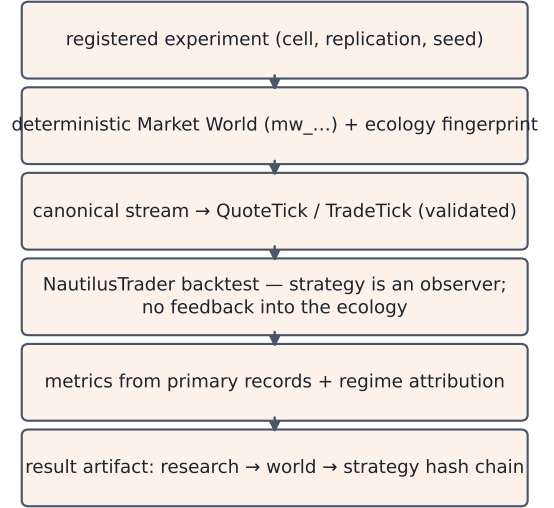


Fig. 6. Market-world export and strategy evaluation. The strategy observes the world; feedback into the ecology is deliberately absent in the current implementation.

Strategy results thus reference an *environment identity*, not a data file.

Worlds export to NautilusTrader’s [22] standard QuoteTick/TradeTick types after strict validation (crossed quotes, non-monotone timestamps, and invalid quantities are rejected), and a hashed reference strategy library (buy-and-hold, EMA crossover, mean reversion) runs in its backtest engine. All metrics derive from primary records (the strategy’s own submission/fill log plus the world’s quote stream): PnL, drawdown, turnover, fill rate, slippage versus prevailing mid, exposure, and *regime-conditioned* breakdowns using Tezcat’s own exported regime intervals. Result identity is the chain research hash → world hash → strategy hash → backtest configuration, plus a recorded environment; reproduction rebuilds the world (hash must match), re-runs the backtest, and requires byte-identical metrics—a NautilusTrader version mismatch is reported as *non-identical* and fails, never silently accepted (Fig. 6).

Two boundaries are explicit. First, the bridge is **backtest-only**: no sandbox or live-trading surface exists. Second, the strategy is an **observer**: its orders fill against the exported stream inside NautilusTrader and *do not feed back* into the Tezcat ecology; market-impact claims are out of scope by construction, and closing this loop is future work gated on an explicit co-simulation contract (Sec. XVII). A further layer (the S5 research control plane) links datasets, distributional forecasts from a reference model, cost-aware portfolio sizing (reference CVaR grid or an optional skfolio adapter [27]), worlds, backtests, and risk reports into a single typed artifact graph with whole-chain hash reproduction; we note it for completeness and audit it in Sec. XIII.

TABLE V
STYLIZED-FACT FEATURES EXTRACTED PER SERIES

Feature	Definition (implementation-aligned)
Return moments	mean, std of $r_t = \log(P_t/P_{t-1})$; skewness
Heavy tails	excess kurtosis; Hill tail index over top-decile $ r_t $
Linear autocorrelation	$\text{acf}(r_t, \ell)$, $\ell \in \{1, 2, 5, 10\}$
Volatility clustering	mean $\text{acf}(r_t , \ell)$ over the same lags
Drawdown	$\max_t \text{DD}_t$
Activity	volume autocorrelation (when volumes exist)

XI. STYLIZED FACTS AND CALIBRATION

The feature extractor follows Cont [4]: return moments, excess kurtosis, Hill tail index, return and absolute-return autocorrelations at fixed lags, a volatility-clustering summary, maximum drawdown, and optional volume clustering (Table V). Comparison is *ensemble-based*: the synthetic side is a ≥ 10 -replication ensemble, and each target feature is reported with the ensemble q05–q95 band, an inside/outside verdict, and a z -distance; a partial match is the expected and informative outcome, not a failure. Calibration is disciplined by construction: calibration and validation sources must differ (identical sources are a schema violation), search is a deterministic, budgeted grid with full evaluation logging and explicit truncation flags, results are frozen, and validation never re-tunes. Two methodological findings are reported rather than hidden. First, a pure $|z|$ objective was evaluated and *rejected* because dividing by ensemble spread rewards diffuse models; the objective is relative error of the ensemble mean, with z retained as a diagnostic. Second, identifiability is real and uneven: tick size (a strong, monotone driver of return volatility) was recovered exactly from a single target series, whereas agent *count* proved weakly identifiable—its effect saturates below single-series noise. Calibration claims are therefore bounded by what the data can actually identify.

XII. DEVOPS AND DEPLOYMENT ARCHITECTURE

Tezcat is container-first and locally deployable; Table VI classifies every component by *in-repository evidence*, per the audit requirement that architecture aspirations must not be reported as experimental claims. Implemented/configured: a Dockerfile packaging backend, CLI, and built frontend; Docker Compose for one-command local orchestration; GitHub Actions workflows for CI and GitHub Pages deployment of the dashboard; a Render blueprint deploying the public backend demo with explicit public-demo guardrails; and an AWS SAM template for optional serverless deployment. Documented/demonstrated: Daytona-based containerized development environments (this paper itself was produced in one) and the public demo runbook. **No Kubernetes manifests, Minikube/Kind configuration, or Jenkins pipelines exist in the repository**; these are future work, and Fig. 7 renders that distinction visually. Containerization also serves reproducibility [20]: environment identity (Python, package, and adapter versions) is recorded on research artifacts and checked during reproduction.

TABLE VI
DEVOPS COMPONENTS BY IN-REPOSITORY EVIDENCE

Component	Status	Evidence
Docker	implemented	Dockerfile
Docker Compose	implemented	docker-compose.yml
GitHub Actions	implemented	workflows ci.yml, deploy-pages.yml
GitHub Pages	implemented	Pages deploy workflow; public-demo docs
Render	configured	render.yaml blueprint; deployment docs
AWS (SAM)	configured	infra/templates/ template.yaml
Daytona	documented/ demonstrated	docs/docker.md; development environment
Kubernetes, Minikube/Kind	future work	no manifests in repository
Jenkins	future work	no pipeline in repository

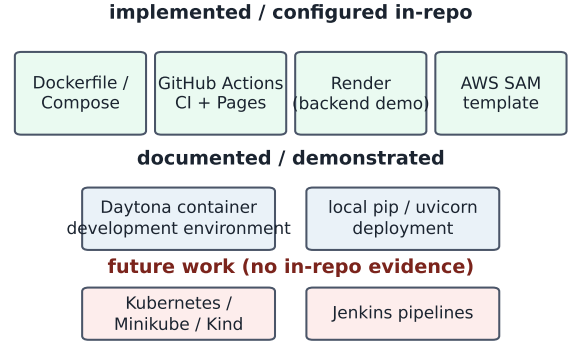


Fig. 7. Deployment components partitioned by in-repository evidence.

A. Security and Data Governance

External access is read-only public market data; no credential is required, stored, or forwarded for any endpoint the adapters use, and any future credentialed integration is confined to server-side environment variables (never experiment JSON, hashes, artifacts, logs, or the frontend). Rate limits are respected by construction; live provider access is disabled by default (offline fixtures only) and gated by an explicit server-side opt-in; public-demo mode additionally restricts imports and caps run sizes. Dataset provenance records license/terms and permitted use. There is no wallet functionality, no live trading, and no financial execution of any kind: Tezcat is a research laboratory, not an execution venue, and its documentation enumerates claims it must never make (e.g., “automated hedge fund,” “predicts crashes”).

XIII. EXPERIMENTAL EVALUATION

All evidence in this section was re-verified at commit 16448b4; no number is carried forward from documentation without re-execution unless explicitly labeled as an archived value.

A. Determinism. The engine-level gate (same seed $\Rightarrow$ identical outputs; different seed $\Rightarrow$ different outputs) passes at the audited commit, and the frozen seed-42 preset baselines (Table VII) remain byte-identical to their F0 records.

B. Reproduction: `tezcat reproduce` re-executed sampled runs of the case-study experiments and verified seeds, state hashes, event hashes, event-log chains, and metrics; the strategy-bridge and research-plane reproducers additionally verified world hashes and full artifact-chain hashes, including byte-level metric equality.

C. Parallel equivalence: The 300-run factorial of Sec. XIV was re-executed with four worker processes; because seeds derive from (root, cell, replication), results are independent of scheduling, and the suite’s differential tests assert serial/parallel byte equality.

D. Fault tolerance: Fault-injection tests cover torn tmp files, crashed writers, duplicate concurrent runners, and provider failures (timeouts, malformed payloads, schema drift, rate limits); the shared-tmp-filename defect found by these tests (Sec. VIII) is fixed and regression-tested.

E. Differential validation: An independent reference matcher is compared against the production matching engine on generated order streams, guarding the microstructure semantics that all higher layers depend on.

F. Stylized facts: Fig. 11 compares a 12-replication stable-ecology ensemble against the seed-42 flash-crash series (a synthetic target; no licensed real-market data is bundled). As expected, tail and clustering features of the crash series fall outside the stable ensemble’s q05–q95 bands while location features overlap—exactly the discriminating behavior the comparison layer is designed to expose.

G. Calibration: The identifiability results of Sec. XI (tick size recovered; agent count weakly identifiable; $|z|$ objective rejected) are enforced by tests and documented as methodological findings.

H. Strategy evaluation: Sec. X’s bridge runs the reference strategies against deterministic worlds with full provenance; Fig. 14 and Table IX report a regime-conditioned example, and reproduction requires byte-identical metrics under a matching NautilusTrader version.

I. External integration: The S3 layer’s adapter, normalization, lineage, quality-rejection, matching, and bridge-to-experiment paths are covered by offline fixture tests, including an end-to-end flow from fixture import to hash-verified reproduction of a signature-derived experiment.

J. Scenario execution: S2 tests compile dashboard scenarios into registered experiments and execute them through TradeOps, confirming the single-execution-path property.

The complete suite comprises **415 tests, all passing** at the audited commit (≈ 37 s wall time in the development container), spanning unit, property, determinism, replay, differential, fault-injection, and integration levels.

A. Preset Demonstrations

The three presets are demonstrations, not evidence (single seed). The flash-crash ecology (whale sell program, market-maker withdrawal, panic sentiment) produces the canonical anatomy in Fig. 9—price collapse with spread expansion and depth evaporation, crisis-regime detection, and recovery—while the momentum-heavy bubble ecology drifts +15.45%

TABLE VII
FROZEN SEED-42 PRESET BASELINES (F0 AUDIT RECORD, VERBATIM;
RE-VERIFIED AT THE AUDITED COMMIT)

Preset	Steps	Return	Max DD	Regimes (% steps)
stable_baseline	1,500	−0.05%	0.10%	stable 100
flash_crash	2,000	+1.85%	29.39%	58.3 / 15.8 / 26.0 (st/cr/rec)
bubble_formation	2,200	+15.45%	17.04%	stable 100

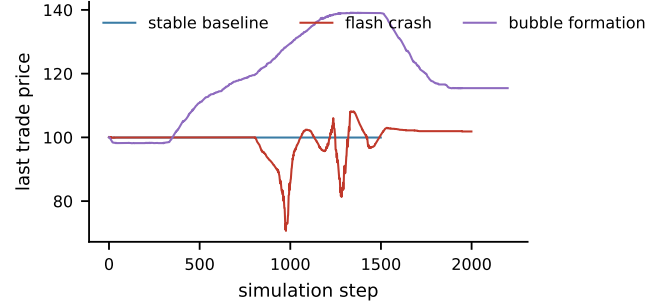


Fig. 8. Seed-42 price trajectories of the three preset ecologies (endogenous prices; single-seed demonstrations).

(Fig. 8). The F0 audit records an instructive definitional honesty case: the bubble run’s “crash” flag is true purely because its 17.04% drawdown exceeds a 15% threshold while the regime engine remains stable throughout—the flag is a drawdown threshold, not a validated crash event. Fig. 10 shows aggregate PnL by agent class for the flash-crash run; the whale’s scripted selling transfers wealth into the surviving classes, with market makers bearing inventory losses (values are from the run report artifact; single-seed, illustrative only).

B. Margin-Spiral A/B (F8)

The committed specification `margin_spiral_ab.json` asks whether the leverage cap determines if a pump-and-dump ends in a liquidation spiral (20 replications per arm, identical shocks and ecology). Re-executed at the audited commit, it re-minted its original research identity (`expv_417305c181af`) and reproduced the archived statistics exactly: tight margin ($2\times$ cap) mean max drawdown 0.0217 (q95 0.0306); loose margin ($10\times$ cap) mean 0.1847 (q95 0.4122); difference $\Delta = 0.163$, 95% CI [0.103, 0.229], Holm-adjusted $p = 0.0005$. Within the synthetic ecology, leverage is an endogenous mechanism producing a liquidation-spiral phenomenology—consistent with, but not evidence about, real-market leverage dynamics [14]–[16].

XIV. CASE STUDY: LEVERAGE $\times$ LIQUIDITY

The centerpiece experiment was conducted *as a user* of the platform, entirely through the public CLI surface, with the hypothesis pre-stated in the committed study document.

A. Design

Question: does increasing leverage amplify the destabilizing effect of declining market liquidity? *Pre-stated hypothesis:* drawdown, forced liquidation, and crash frequency rise with

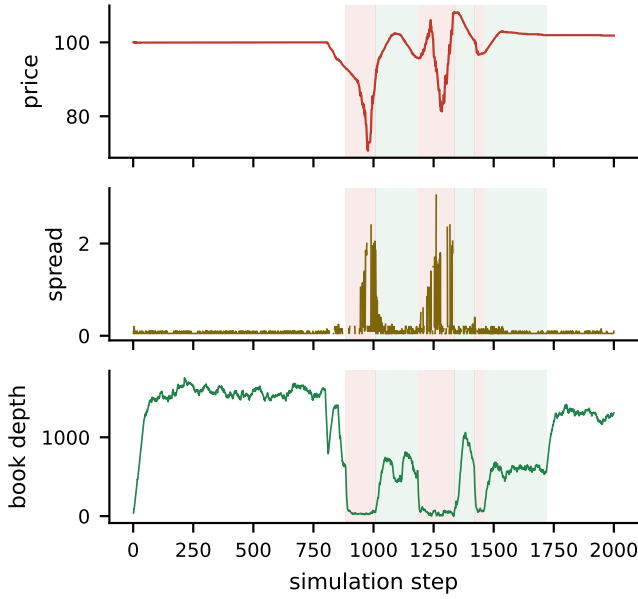


Fig. 9. Flash-crash anatomy (seed 42): last price, quoted spread, and total book depth; shaded bands mark detected crisis (red) and recovery (green) regimes.

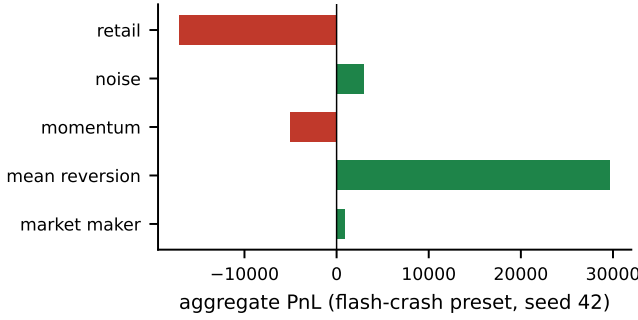


Fig. 10. Aggregate PnL by agent class, flash-crash preset (seed 42; from the persisted run report).

leverage, and rise *faster* when market-maker liquidity is thin (a positive leverage $\times$ illiquidity interaction). Two committed specifications over the pump-then-dump margin ecology: a 5×3 factorial map (leverage cap $\{2, 4, 6, 8, 10\}\times$ via initial margin, market-maker count $\{1, 3, 5\}$, 20 replications per cell, 300 runs) and a 2×2 extremes design ($\{2\times, 10\times\}\times\{1, 5\}$ market makers, 30 replications per cell, 120 runs)—420 runs total. Primary metric: maximum drawdown; secondary: crash frequency (fraction of seeds with `crash_detected`), forced-liquidation volume, realized volatility.

B. Results

Both experiments re-minted their archived research identities at the audited commit (`expv_60fabaa6f7ca`, `expv_cfc17774801a`; the map’s full research hash begins `60fabaa6f7ca8453...`), and the statistics below are from that re-execution.

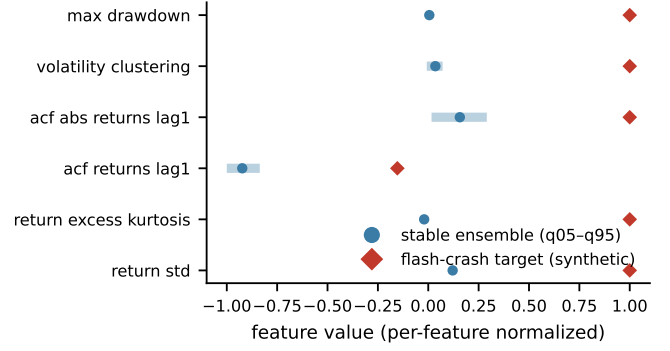


Fig. 11. Stylized-facts comparison: stable-ecology ensemble bands (q05–q95, $n=12$) versus the seed-42 flash-crash series as a synthetic target. Per-feature normalization for display; underlying values from the feature extractor.

1) *Leverage: confirmed, threshold-like*: Pooled over liquidity, mean maximum drawdown rises from 0.022 at $2\times$ to 0.063 at $4\times$ and 0.132–0.148 at $6\text{--}10\times$; crash frequency is **0/60** at $2\times$ across all liquidity levels, rising to 20–65% per cell at $8\text{--}10\times$ (Fig. 12, Fig. 13, Table VIII). The 2×2 main effect of leverage is +0.113 maximum drawdown, replicating and extending the F8 result.

2) *The pre-registered interaction: rejected*: At the extremes, the leverage $\times$ illiquidity interaction contrast is

$$+0.0146, \quad 95\% \text{ CI } [-0.0233, +0.0567],$$

spanning zero, and the liquidity main effect is +0.004—essentially nothing. Thin liquidity does *not* measurably amplify leverage-driven drawdowns in this model; the hypothesis is rejected as stated. We regard this rejection, produced and reproduced by the platform’s own machinery, as a demonstration that the system supports falsification rather than confirmation.

3) *The surprise: liquidity as transmission channel*: Forced-liquidation volume concentrates in the *deeper* books (Table VIII); thin books exhaust, forced sells fail to fill, and the cascade self-limits; deeper books let deleveraging actually transact, transmitting price impact into other levered agents’ collateral. The worst observed cell is $8\times$ leverage with *medium* liquidity (3 market makers): 65% crash frequency and the largest mean forced volume (10.5). Because forced volume measures *fills*, *not intent* (Sec. V), this mechanism is directly readable from the artifacts. **Within the Tezcat ecology and this experimental configuration**, market-maker liquidity under high leverage behaves as a transmission channel for deleveraging rather than simply a stabilizing cushion—an interaction structure reminiscent of, but not evidence for, liquidity-spiral accounts of real markets [16], [17].

4) *Design lessons*: The effect is non-monotone in liquidity at high leverage, which the 2×2 extremes design is structurally unable to detect—the medium-liquidity spike lives exactly where extremes-only designs do not look. Crash-frequency cells use $n=20$, so the 65% vs. 20% contrast carries wide binomial uncertainty; the archived study explicitly marks the $8\times/3\text{-MM}$ spike as suggestive and calls for a higher- n repli-

TABLE VIII

LEVERAGE $\times$ LIQUIDITY MAP (RE-EXECUTED AT THE AUDITED COMMIT;
 $n=20$ PER CELL): MEAN MAX DRAWDOWN / CRASH FREQUENCY / MEAN
 FORCED VOLUME FILLED

Leverage	1 MM	3 MM	5 MM
2 $\times$	.025 / 0% / 0	.020 / 0% / 0	.021 / 0% / 0
4 $\times$	.083 / 15% / 0	.064 / 10% / 0	.043 / 0% / 0
6 $\times$	.113 / 30% / 0.1	.132 / 25% / 3.0	.151 / 30% / 6.4
8 $\times$	.110 / 20% / 0.0	.241 / 65% / 10.5	.093 / 20% / 0.0
10 $\times$	.133 / 30% / 1.8	.171 / 50% / 3.1	.137 / 40% / 5.5

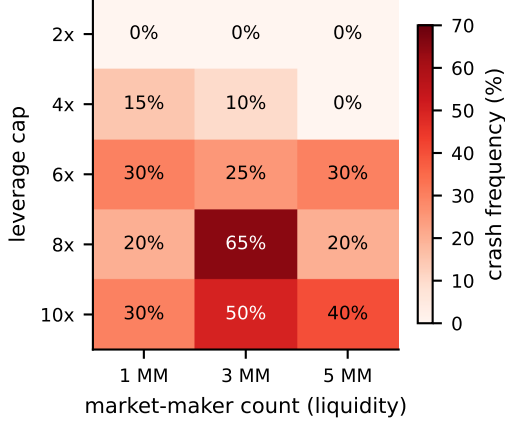


Fig. 12. Crash frequency across the leverage $\times$ liquidity map (300 runs, $n=20$ per cell, re-executed at the audited commit). The maximum sits at 8 $\times$ leverage with *medium* liquidity.

cation of that row. Both caveats are as much results about experimental design as about the model.

C. Regime-Conditioned Strategy Example

To illustrate the S4 bridge end to end, a flash-crash world was built from a registered preset experiment and the EMA-crossover reference strategy was evaluated against it (Table IX, Fig. 14). The world’s fingerprint records 64.3% stable, 7.0% crisis, and 28.8% recovery occupancy with a detected crash; mean fill slippage versus mid in the crisis regime (0.100) is 3.4 $\times$ its stable-regime level (0.029)—the execution-cost signature of the engineered liquidity collapse, measured by the bridge’s regime attribution rather than reconstructed after the fact. Single world, single strategy: an illustration of the measurement surface, not a strategy claim.

XV. DISCUSSION

Three observations organize the evidence. First, the platform’s scientific value materialized exactly where its design intended: the leverage $\times$ liquidity study produced a *rejected* pre-registered hypothesis and an unanticipated mechanism (transmission-channel liquidity) that was diagnosable only because forced liquidation is real order flow and every intermediate quantity is artifact-backed. A system optimized for confirmation would have reported the leverage effect and stopped. Second, reproducibility compounds: because experiment identity is content-addressed over configuration,

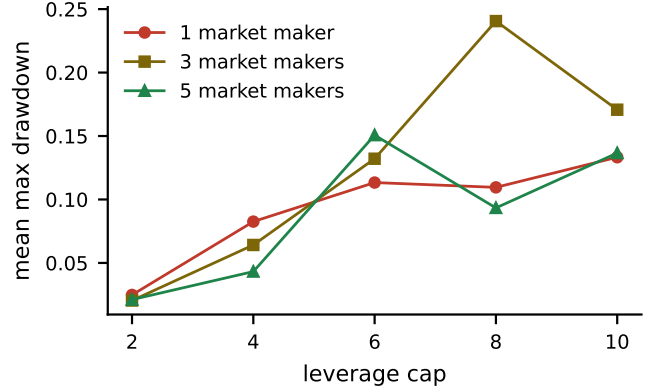


Fig. 13. Mean maximum drawdown versus leverage cap by market-maker count (same 300 runs).

TABLE IX

STRATEGY-EVALUATION EXAMPLE: EMA CROSS ON A FLASH-CRASH
 WORLD (SINGLE SEED; ILLUSTRATIVE)

total return	+0.096%	orders / fill events	158 / 159
max drawdown	0.064%	turnover	1.57M
slippage vs. mid (mean)	0.029		
By regime (PnL / fills / mean slippage)			
stable	−535.0 / 102 / 0.029		
crisis	+540.1 / 2 / 0.100		
recovery	+950.0 / 55 / 0.026		

schema, seed plan, and code version, specifications committed several phases earlier re-minted identical research hashes at the audited commit—turning “the result still stands” from a claim into a checkable property. Third, the outward bridges preserve epistemic boundaries by construction: external observations are proxies that inform design; synthetic worlds are environments with identities; strategies are observers with hashed configurations. Each boundary is enforced by types, tests, and language in the artifacts themselves, which we consider as much a contribution as any single mechanism.

XVI. LIMITATIONS AND THREATS TO VALIDITY

(1) The synthetic ecology is not a real market; all findings are statements about a specified model. (2) Agent behavioral models are deliberately simple; richer cognition could change dynamics qualitatively. (3) Calibration can be non-identifiable; the platform demonstrated this concretely (agent count), and any calibration claim is bounded by identifiability. (4) External event-market integration establishes observation, not causation; divergence and lead/lag metrics are observational. (5) Prediction-market prices are market-implied probability proxies, not ground truth [23]. (6) No licensed real-market dataset is bundled; stylized-fact comparisons in this paper use synthetic targets, and real-data validation remains open. (7) The NautilusTrader bridge treats strategies as observers; absent feedback, market-impact and crowding questions are out of scope. (8) Synthetic worlds reproduce selected properties, not all properties, of real markets. (9) Reported wall-times are

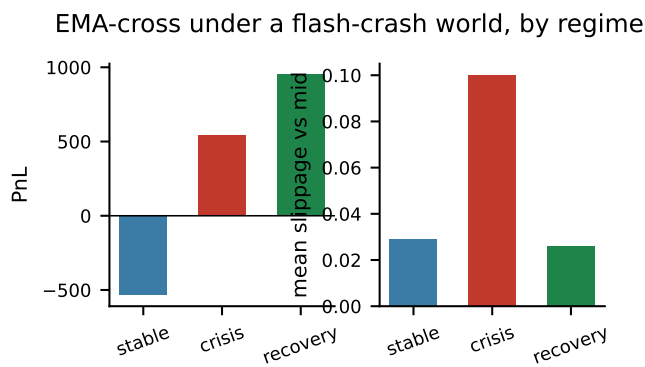


Fig. 14. Regime-conditioned strategy metrics for the example of Table IX.

environment-dependent; the determinism guarantees are not, but performance figures should not be quoted as platform constants. (10) Several conclusions (e.g., the transmission-channel result) are configuration-dependent: single asset, one venue, market-order liquidation, no funding network. (11) Causal language in this paper is strictly intra-model; higher-order causal inference (mediation, dose-response designs) requires further experiment types. (12) Crash detection is a drawdown threshold, not a validated event definition—an F0-documented definitional limitation.

XVII. FUTURE RESEARCH

(a) Bidirectional Tezcat $\leftrightarrow$ NautilusTrader co-simulation in which a strategy becomes a participant in the ecology, gated on an explicit synchronization contract (time authority, same-timestamp event ordering, latency modeling, order-injection boundary, feedback timing, checkpointing, and a coupled reproducible identity)—no implementation before that contract is specified. (b) Richer agent cognition (learning, heterogeneous horizons) with the same provenance guarantees. (c) Multi-asset ecologies and cross-asset contagion, including funding networks that would directly re-test the transmission-channel finding. (d) Rigorous empirical calibration against licensed real datasets, with block-bootstrap uncertainty on the target side and ABC/history-matching methods. (e) Distributed Monte Carlo beyond single-machine workers, and GPU acceleration only where it preserves bit-level determinism. (f) Richer microstructure (latency, queue position, fees, multiple venues). (g) Event-market driven scenario generation at scale: standing pipelines from Kalshi/Polymarket event signatures to controlled experiment proposals under human approval. (h) Higher- n replication of the $8\times$ leverage row and formal tests for the non-monotone liquidity effect. (i) A natural-language hypothesis compiler and an AI research-analyst layer are *future work*: no LLM component exists in the current system, and the documented policy for any future analyst confines it to reading artifacts and proposing hypotheses behind human approval, never entering the deterministic measurement loop. (j) Synthetic-world benchmark suites and research-artifact

publication infrastructure (e.g., archived experiment bundles with DOIs).

XVIII. CONCLUSION

Tezcat demonstrates that the standards of experimental science—pre-registered designs, replication, statistical inference, immutable identity, provenance, and reproduction—can be engineered directly into a market-simulation platform rather than appended to it. The platform’s own flagship study behaved the way science should: a large main effect was confirmed, a pre-registered interaction was rejected, and an unanticipated mechanism emerged that was traceable to a specific modeling decision (liquidation as real order flow) through artifact-backed evidence, and the entire chain reproduces from content-addressed identities at a later commit. The system now spans observation (read-only event markets), synthesis (controlled market worlds), and evaluation (strategy backtests), with one provenance thread connecting them. Tezcat predicts nothing about real markets; its contribution is a laboratory in which mechanism hypotheses about markets become falsifiable, replicable, and reproducible experiments.

REPRODUCIBILITY STATEMENT

All experiments in this paper are reproducible from the repository at commit 16448b45c1c4 (415/415 tests passing). The complete loop is:

```
tezcat run examples/leverage_liquidity_map.json
tezcat analyze expv_60fabaa6f7ca
tezcat report expv_60fabaa6f7ca -o report.md
tezcat reproduce 60fabaa6
```

reproduce re-executes sampled runs and verifies seeds, state hashes, event hashes, event-log chains, and metrics against stored artifacts; the analogous verbs exist for market worlds (tezcat lab reproduce) and full research-plane chains (tezcat plane reproduce), both of which fail loudly on dependency-version mismatch. Figures in this paper were generated by the committed scripts in paper/figures/; the paper build is one command (paper/build.sh). The repository README declares an MIT license; no DOI or archived research-artifact identifier exists at the audited commit, and none is claimed.

REFERENCES

- [1] W. B. Arthur, J. H. Holland, B. LeBaron, R. Palmer, and P. Tayler, “Asset pricing under endogenous expectations in an artificial stock market,” in *The Economy as an Evolving Complex System II*. Reading, MA: Addison-Wesley, 1997, pp. 15–44.
- [2] J. D. Farmer and D. Foley, “The economy needs agent-based modelling,” *Nature*, vol. 460, pp. 685–686, 2009.
- [3] C. H. Hommes, “Heterogeneous agent models in economics and finance,” in *Handbook of Computational Economics*. Elsevier, 2006, vol. 2, pp. 1109–1186.
- [4] R. Cont, “Empirical properties of asset returns: stylized facts and statistical issues,” *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.
- [5] A. S. Kyle, “Continuous auctions and insider trading,” *Econometrica*, vol. 53, no. 6, pp. 1315–1335, 1985.
- [6] L. R. Glosten and P. R. Milgrom, “Bid, ask and transaction prices in a specialist market with heterogeneously informed traders,” *Journal of Financial Economics*, vol. 14, no. 1, pp. 71–100, 1985.
- [7] T. Lux and M. Marchesi, “Scaling and criticality in a stochastic multi-agent model of a financial market,” *Nature*, vol. 397, pp. 498–500, 1999.

- [8] B. LeBaron, "Agent-based computational finance," in *Handbook of Computational Economics*. Elsevier, 2006, vol. 2, pp. 1187–1233.
- [9] L. Tesfatsion, "Agent-based computational economics: a constructive approach to economic theory," in *Handbook of Computational Economics*. Elsevier, 2006, vol. 2, pp. 831–880.
- [10] D. K. Gode and S. Sunder, "Allocative efficiency of markets with zero-intelligence traders: market as a partial substitute for individual rationality," *Journal of Political Economy*, vol. 101, no. 1, pp. 119–137, 1993.
- [11] J.-P. Bouchaud, M. Mézard, and M. Potters, "Statistical properties of stock order books: empirical results and models," *Quantitative Finance*, vol. 2, no. 4, pp. 251–256, 2002.
- [12] R. Cont, S. Stoikov, and R. Talreja, "A stochastic model for order book dynamics," *Operations Research*, vol. 58, no. 3, pp. 549–563, 2010.
- [13] C. Chiarella and G. Iori, "A simulation analysis of the microstructure of double auction markets," *Quantitative Finance*, vol. 2, no. 5, pp. 346–353, 2002.
- [14] J. Geanakoplos, "The leverage cycle," *NBER Macroeconomics Annual*, vol. 24, pp. 1–65, 2010.
- [15] T. Adrian and H. S. Shin, "Liquidity and leverage," *Journal of Financial Intermediation*, vol. 19, no. 3, pp. 418–437, 2010.
- [16] M. K. Brunnermeier and L. H. Pedersen, "Market liquidity and funding liquidity," *Review of Financial Studies*, vol. 22, no. 6, pp. 2201–2238, 2009.
- [17] A. Shleifer and R. Vishny, "Fire sales in finance and macroeconomics," *Journal of Economic Perspectives*, vol. 25, no. 1, pp. 29–48, 2011.
- [18] R. D. Peng, "Reproducible research in computational science," *Science*, vol. 334, no. 6060, pp. 1226–1227, 2011.
- [19] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten simple rules for reproducible computational research," *PLoS Computational Biology*, vol. 9, no. 10, p. e1003285, 2013.
- [20] C. Boettiger, "An introduction to Docker for reproducible research," *ACM SIGOPS Operating Systems Review*, vol. 49, no. 1, pp. 71–79, 2015.
- [21] D. Byrd, M. Hybinette, and T. H. Balch, "ABIDES: towards high-fidelity multi-agent market simulation," in *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS)*, 2020, pp. 11–22.
- [22] Nautech Systems, "NautilusTrader: a high-performance algorithmic trading platform and event-driven backtester," https://github.com/nautechsystems/nautilus_trader, version 1.221.0; accessed 2026-08-27.
- [23] J. Wolfers and E. Zitzewitz, "Prediction markets," *Journal of Economic Perspectives*, vol. 18, no. 2, pp. 107–126, 2004.
- [24] K. J. Arrow, R. Forsythe, M. Gorham, R. Hahn, R. Hanson, J. O. Ledyard, S. Levmore, R. Litan, P. Milgrom, F. D. Nelson, G. R. Neumann, M. Ottaviani, T. C. Schelling, R. J. Shiller, V. L. Smith, E. Snowberg, C. R. Sunstein, P. C. Tetlock, P. E. Tetlock, H. R. Varian, J. Wolfers, and E. Zitzewitz, "The promise of prediction markets," *Science*, vol. 320, no. 5878, pp. 877–878, 2008.
- [25] Kalshi Inc., "Kalshi trade API documentation," <https://docs.kalshi.com>, accessed 2026-08-23.
- [26] Polymarket, "Polymarket developer documentation (Gamma, CLOB, and Data API)," <https://docs.polymarket.com>, accessed 2026-08-23.
- [27] skfolio developers, "skfolio: portfolio optimization built on top of scikit-learn," <https://skfolio.org>, version 1.0.0; accessed 2026-08-27.